

04 - Spanning Tree construction

Uno **spanning tree**, o **albero ricoprente**, T di un grafo $G = (V, E)$ è un sottografo **connesso e aciclico** di G tale che $T = (V, E')$ con $E' \subseteq E$.

Come per il protocollo di broadcasting, è spesso utile riuscire a calcolare uno spanning tree su una rete in modo distribuito. La motivazione principale riguarda le operazioni di broadcasting, che fatte su una sottorete risultano più efficienti. In particolare, **costruendo uno spanning tree la singola operazione di broadcast fatta sull'albero costa $n - 1$ invece di $O(m)$ sulla rete originale**.

Spanning Tree construction problem

Lo **Spanning Tree Construction** problem consiste nel costruire, mediante una procedura distribuita, un qualsiasi *spanning tree* T (non necessariamente il minimo), data una rete G . Sia $\text{tree-neig}(x) \subseteq N(x)$, dove ovviamente $x \in V$.

Il problema è formalizzato dalla tripla $P = \langle P_{init}, P_{final}, R \rangle$, dove

- $P_{init} = \forall x \in V : \text{tree-neig}(x) = \{\} \wedge \exists! x \in V : \text{status}(x) = \text{initiator} \wedge \forall y \neq x : \text{status}(y) = \text{idle}$;
- $P_{final} = \bigcup_{x \in V} \text{tree-neig}(x)$ **forma uno spanning tree della rete G : tutti i nodi $x \in V$ hanno nell'insieme $\text{tree-neig}(x)$ tutti e soli i nodi vicini nello spanning tree ottenuto dal protocollo:**

$$\text{tree-neig}(x) = \{y \in N(x) : (x, y) \in E'\}$$

e $\forall x \in V : \text{status}(x) = \text{done}$.

- $R_{step} = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \\ \text{Unique Initiator (UI)} \end{cases}$

A differenza di algoritmi centralizzati, come l'algoritmo di Kruskal per la costruzione del minimo spanning tree, nel mondo distribuito la conoscenza dello spanning tree che viene costruito è divisa tra i vari nodi della rete. **Ogni nodo quindi conosce solamente la parte locale di T a lui adiacente, e non esiste nessun nodo che ha una visione globale dello spanning tree costruito.**

Single Source Shout Protocol

Sia x un generico nodo all'interno del sistema. Allora tale nodo deve decidere chi saranno i suoi vicini nell'albero ricoprente. Per fare ciò, **chiede ai propri vicini se appartengono già allo spanning tree che si sta costruendo, e in base alla risposta il nodo agisce di conseguenza.** Sia y un nodo vicino ad x , formalmente $y \in N(x)$, allora

- se y appartiene già allo spanning tree allora si scarta;
- se y ancora non appartiene allo spanning tree allora x propone ad y di diventare un suo nodo figlio.

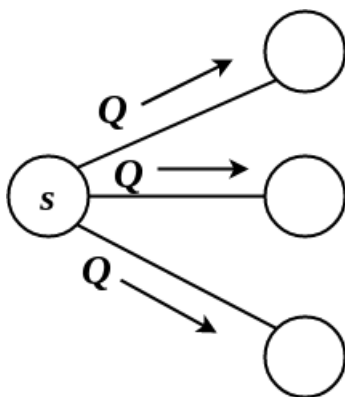
In generale, per via dei ritardi di trasmissione, un nodo può ricevere molteplici richieste di diventare nodo figlio. In questo caso, risponde con YES alla prima proposta ricevuta e NOU a tutte le altre.

In particolare, quando il nodo y diventa figlio di un nodo x nello spanning tree, invia in broadcast a tutti i suoi vicini, eccetto il nodo padre, la proposta di diventare nodo figlio.

Inoltre ogni nodo alla ricezione di ogni richiesta deve rispondere con YES o NOU.

Dunque il protocollo **Shout** risulta essere un broadcast con dei messaggi di acknowledgement.

casi sorgente e nodo x .



Si definisce l'insieme degli stati del processo come segue

$$S = \{\text{init, idle, active, done}\}$$

dove $S_{init} = \{\text{init, idle}\}$ è l'insieme degli stati iniziali e $S_{final} = \{\text{done}\}$ è l'insieme degli stati finali.

L'idea è che inizialmente tutti i nodi tranne l'iniziatore si trovano nello stato idle. L'iniziatore nello stato initiator si sveglia da un impulso esterno al sistema e invia i messaggi di domanda, indicati con Q , a tutti i suoi vicini. Con il messaggio Q un generico nodo x propone ai nodi a lui adiacenti se vogliono essere connessi a x , cioè essere figli di x , all'interno dello spanning tree. Quando un nodo y nello stato idle riceve un messaggio Q , questo si attiva, sceglie come padre il primo nodo da cui ha ricevuto tale messaggio (si ricorda che per via dei ritardi un nodo può ricevere più messaggi Q contemporaneamente) e invia a tale nodo il messaggio YES (Y). Successivamente, y invia il messaggio Q a tutti i nodi a lui adiacenti, tranne il nodo che l'ha attivato, e cambia lo stato in active. Un nodo x nello stato active ha già scelto un padre, dunque risponde sempre NOU (N) a tutti i messaggi di tipo Q che riceve. Se invece riceve un messaggio di tipo Y , ovvero se è stato scelto come padre da un nodo a lui adiacente, allora aggiunge tale nodo in $\text{tree-neig}(x)$. Ogni nodo conta le risposte ricevute dai nodi adiacenti dopo aver inviato Q , comprendendo

in tale conteggio anche il primo messaggio di tipo Q ricevuto dal padre (il contatore parte da 1 per ogni agente non initiator): quando il contatore diventa pari al grado del nodo, esso termina e entra nello stato done.

Si formalizza quanto descritto: il protocollo SHOUT è il seguente.

```
if state == "INIT":
    spontaneously:
        root = True
        tree_neig = {}
        send(Q) to neighbors
        counter = 0
        state = "ACTIVE"

if state == "IDLE":
    receiving(Q):
        root = False
        parent = sender
        tree_neig = {sender}
        send("YES") to sender
        counter = 1

        if counter == len(neighbors):
            state = "DONE"
        else:
            send(Q) to neighbors - {sender}
            state = "ACTIVE"

if state == "ACTIVE":
    receiving(Q):
        send("NO") to sender
    receiving("YES"):
        tree_neig = tree_neig + {sender}
        counter += 1
        if counter == len(neighbors):
            state == "DONE"
    receiving("NO"):
        counter += 1
        if counter == len(neighbors):
            state == "DONE"

if state == "DONE":
    None
```

La rete costituita da tutti gli agenti in V e dagli archi attraverso i quali è passato un messaggio Y andranno a formare un albero ricoprente di G .

Terminazione

Il protocollo SHOUT è molto simile al protocollo FLOOD, con l'aggiunta di messaggi di *feedback* Y e N . Dato che siamo sotto l'assunzione che di *connectivity* e *total reliability*, allora certamente ogni nodo x (eccetto la sorgente) riceverà, entro un tempo finito, almeno una richiesta Q di diventare figlio. Perciò ogni nodo nello stato *idle* passerà allo stato *active*, con il proprio contatore pari a 1. Per ogni richiesta ricevuta tutti i nodi risponderanno Y o N . Dato che prima di entrare nello stato *done* un nodo deve ricevere una risposta da tutti i vicini, e dato che per costruzione tutti gli agenti rispondono a tutte le richieste ricevute e inoltre, sempre per la *total reliability*, i messaggi inviati giungeranno alla propria destinazione entro un tempo finito, allora prima o poi tutti i nodi passeranno dallo stato *active* allo stato *done*, terminando localmente il proprio task: il protocollo termina quindi anche globalmente.

Correttezza

Si dimostra che la rete formata dall'unione di tutti gli insiemi $\text{tree-neig}(x)$ è uno *spanning tree* per G .

Coerenza

Per prima cosa si dimostra la **coerenza** degli insiemi $\text{tree-neig}(x)$: per ogni nodo x , se x ha come vicino y nello *spanning tree*, cioè $y \in \text{tree-neig}(x)$, allora anche x deve essere vicino di y nello *spanning tree*, cioè $x \in \text{tree-neig}(y)$.
Si vuole mostrare quindi che, dati $x, y \in V$:

$$y \in \text{tree-neig}(x) \iff x \in \text{tree-neig}(y)$$

Certamente uno dei due nodi x e y deve essere il nodo padre dell'altro; sia, senza perdere di generalità, x il nodo padre di y . Per ipotesi, $y \in \text{tree-neig}(x)$ e y è figlio del nodo x : allora y ha risposto Y alla proposta di x . Ma se y ha risposto Y , allora y era nello stato *idle* alla ricezione della proposta di x e per definizione del protocollo y inserisce x all'interno del suo insieme $\text{tree-neig}(y)$.

Connessione

A questo punto si dimostra che l'unione di tutti gli insiemi $\text{tree-neig}(x)$ formano una rete **connessa**, ossia, si mostra che per ogni nodo x diverso dalla sorgente, se x ha risposto Y ad un nodo y , allora x si trova nel *tree-neighbour* di y , dove quest'ultimo nodo è connesso alla sorgente s tramite una sequenza di archi sui quali sono passati i messaggi di conferma Y . Tale sequenza, assieme all'arco (x, y) che si trova nell'albero costruito T per via di quest'ultima interazione, formerà un cammino da s ad x .

Sia x un nodo qualsiasi, diverso dalla sorgente s . Se x ha risposto Y ad un nodo y , per definizione del protocollo allora $x \in \text{tree-neig}(y)$, e dato che y deve aver inviato una proposta al nodo x , y si trova nello stato *active* alla ricezione della risposta Y da x . Ma y invia una proposta al nodo x se e solo se:

- y è la sorgente s ;

- y ha ricevuto una proposta da parte di un nodo z alla quale ha risposto Y , e dunque $z \in \text{tree-neig}(y)$ e $y \in \text{tree-neig}(z)$;
Iterando questo processo fino alla sorgente s , si ottiene proprio un cammino che va dalla sorgente s fino ad x tramite una sequenza di archi sui quali è passato il messaggio Y .

Coerenza e connessione implicano che la sottorete così costruita ricopre G , ossia, che $\bigcup_{x \in V} \text{tree-neig}(x) = V$ e $\forall x, y \in V, \exists p(x, y) \subseteq E$ tale che $\forall (u, v) \in p(x, y)$ è passato un messaggio Y .

Aciclicità

Infine si dimostra che l'unione dei tree-neighbour formano una rete **aciclica**. Questo si può inferire dal fatto che ogni nodo, eccetto la sorgente, invia **una sola** risposta Y .

Si assuma che il grafo T indotto dalla relazione di parentela che caratterizza il protocollo contenga un ciclo diretto $x_0, x_1, \dots, x_{k-1}$, ossia, x_i è padre di x_{i+1} in T . Questo ciclo non può contenere l'initiator s , perché non invia nessun messaggio Y . Per la dimostrazione della connettività, in T deve esistere un cammino da s verso ogni altro nodo, inclusi quelli nel ciclo. Ciò implica che in T vi sarà un nodo y non presente nel ciclo ($y \neq x_i \forall i = 0, \dots, k-1$), il quale risulterà connesso ad un nodo x_i nel ciclo. Questo vuol dire che x_i ha inviato un messaggio Y ad y , ma essendo x_i un nodo nel ciclo, allora, x_i deve aver mandato un messaggio Y ad x_{i-1} . Ma questo è impossibile perché un'entità non invia più di un messaggio Y , assurdo.

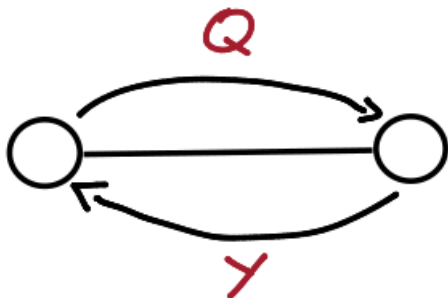
In conclusione, avendo mostrato che il protocollo identifica una sottorete T che ricopre G , la quale risulta essere connessa e aciclica, allora T è un albero ricoprente di G .

Message Complexity

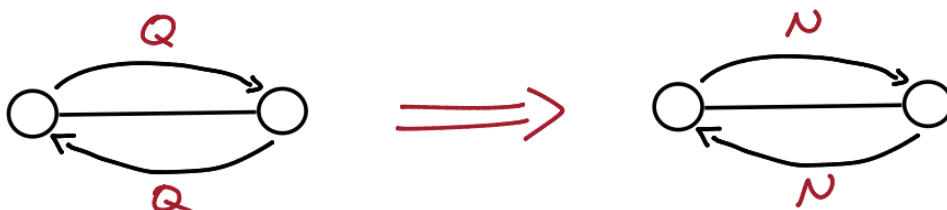
Si discute ora la message complexity del protocollo SHOUT. Si osserva che i messaggi inviati dal protocollo possono essere divisi in due categorie: il messaggio Q che viene inviato da un nodo x per proporre ad un nodo y di diventare un suo nodo figlio nello spanning tree, e i messaggi Y e N che vengono inviati da y in risposta al messaggio Q .

Si effettua l'analisi dei messaggi che percorrono gli archi della rete:

- Per ogni arco che farà parte dello spanning tree costruito dal protocollo passano due messaggi: da un lato passa il messaggio Q e dall'altro passa la risposta affermativa Y . Dato che questi archi sono esattamente $n - 1$, allora vengono inviati $n - 1$ messaggi di tipo Q ed $n - 1$ messaggi di tipo Y .

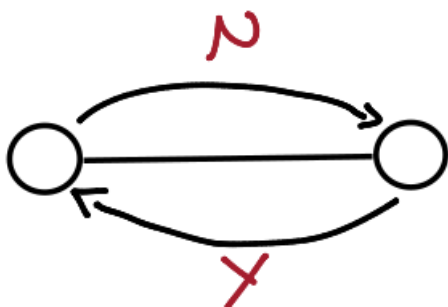


- Per ogni arco che non farà parte dello spanning tree passano quattro messaggi: due messaggi di tipo Q in entrambe le direzioni, seguiti da due messaggi di risposta negativa N in entrambe le direzioni. Questi archi sono esattamente $m - (n - 1)$.

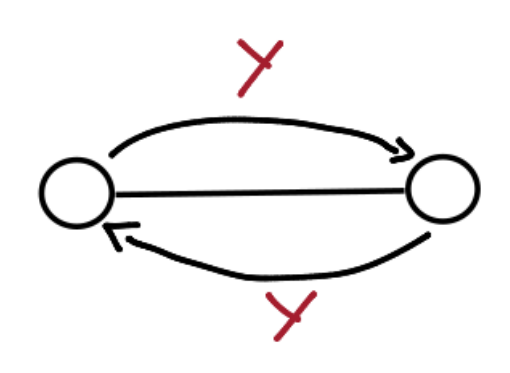


Si osserva su ogni arco non potranno mai passare le coppie di messaggi N, Y ed Y, Y :

- N, Y non può accadere in quanto il nodo sinistro per rispondere N deve prima aver ricevuto una richiesta dal nodo destro, ma per inviare una richiesta il nodo destro deve avere già un padre, dunque non può rispondere nuovamente Y ;



- Y, Y non può accadere in quanto due nodi per inviarsi reciprocamente Y devono essersi prima inviati reciprocamente Q , ma ciò implica che hanno già un padre e che quindi hanno già inviato Y a un altro nodo.



Tornando all'analisi del numero di messaggi, si ottiene che

- **Totale messaggi** Q : uno per ogni arco nello spanning tree e due per tutti gli altri archi, dunque $n - 1 + 2(m - (n - 1)) = 2m - n + 1$;
- **Totale messaggi** Y : uno per ogni arco nello spanning tree, dunque $n - 1$;
- **Totale messaggi** N : due per ogni arco che non fa parte dello spanning tree, dunque $2(m - (n - 1))$.

In conclusione, si ottiene

$$\begin{aligned} M(\text{SHOUT}) &= (2m - n + 1) + (n - 1) + 2(m - (n - 1)) \\ &= 4m - 2n + 2 \end{aligned}$$

Si osserva che, come già fatto notare, il protocollo SHOUT effettua un flooding dei messaggi Q seguiti da una risposta del tipo Y o N . Dunque segue che

$$M(\text{SHOUT}) = 2 \cdot M(\text{FLOOD})$$

In accordo con l'analisi appena fatta.

Protocollo Shout+

Si osserva che nel protocollo SHOUT i messaggi di risposta negativa N non sono necessari per far sapere al nodo che ha inviato la richiesta quando entrare nello stato done. Infatti, se un nodo y invia N ad x , allora y ha inviato il messaggio Q ad x prima dell'invio di N , in quanto y per inviare N deve trovarsi nello stato active, e per trovarsi nello stato active il nodo y deve avere già un nodo padre e aver mandato il messaggio Q a tutti i suoi vicini. Dunque il nodo x può interpretare il messaggio Q inviato da y come il messaggio N e non inviare la proposta ad y .

Tale idea viene implementata nel protocollo seguente, che prende il nome di SHOUT+.

```
if state == "INIT":
    spontaneously:
        root = True
        tree_neig = {}
        counter = 0
```

```

    send(Q) to neighbors
    state = "ACTIVE"

if state == "IDLE":
    receiving(Q):
        root = False
        parent = sender
        tree_neig = {sender}
        counter = 1

        send("YES") to sender

        if counter == len(neighbors):
            state = "DONE"
        else:
            send(Q) to neighbors - {sender}
            state = "ACTIVE"

if state == "ACTIVE":
    receiving(Q):
        counter += 1
        if counter == len(neighbors):
            state == "DONE"
    receiving("YES"):
        tree_neig = tree_neig + {sender}
        counter += 1
        if counter == len(neighbors):
            state = "DONE"

if state == "DONE":
    None

```

Nel protocollo SHOUT+ non ci sono più messaggi di tipo N . In ogni caso il comportamento è analogo a quello analizzato prima, in quanto il messaggio Q contiene tutte le informazioni che portava il messaggio N .

Message complexity

Si effettua un'analisi del numero dei messaggi che attraversano i link, per determinare la message complexity:

- Per ogni arco che farà parte dello spanning tree costruito dal protocollo passano due messaggi: da un lato passa il messaggio Q e dall'altro passa la risposta affermativa Y . Dato che questi archi sono esattamente $n - 1$, allora vengono inviati $n - 1$ messaggi di tipo Q ed $n - 1$ messaggi di tipo Y .
- Per ogni arco che non farà parte dello spanning tree vengono scambiati due messaggi di tipo Q . Questi archi sono esattamente $m - (n - 1)$.

Dunque risulta una message complexity pari a

$$M(\text{SHOUT}+) = 2(n-1) + 2(m - (n-1)) = 2m$$

Terminazione globale nel protocollo SHOUT+

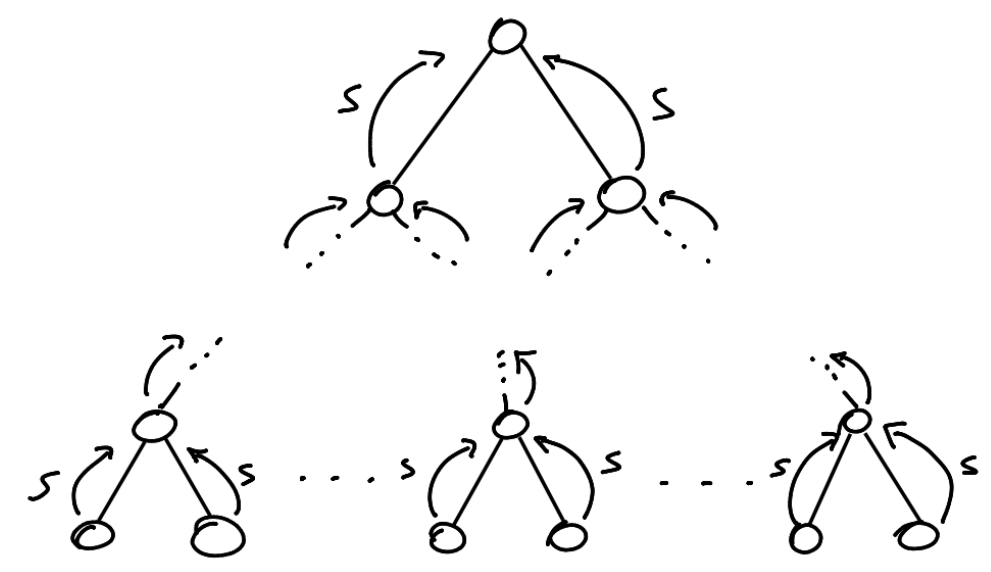
Si osserva che per come sono stati definiti i protocolli SHOUT e SHOUT+, si ottiene una *terminazione locale*, ovvero ogni nodo sa quando il proprio compito è terminato, ma nessun nodo è in grado di stabilire a quale istante t l'intero spanning tree è costruito, cioè a quale istante t tutti i nodi si trovano nello stato DONE (si ricorda che t esiste finito).

Tramite una modifica del protocollo è possibile ottenere la **terminazione globale**, in cui ogni nodo sa esattamente quando il protocollo è terminato e quando lo spanning tree è stato costruito.

L'idea è quella di far partire una operazione di *accumulation* bottom-up, a partire quindi dalle foglie, che si sparge verso l'alto nello spanning tree costruito per arrivare infine alla sorgente s . In particolare, si aggiunge un nuovo messaggio S al protocollo, il quale viene condiviso nel modo seguente:

- I nodi foglia inviano S al proprio padre;
- Un nodo non foglia invia la proprio padre il messaggio S solo dopo aver ricevuto S da tutti i propri figli;

Si osserva che i nodi foglia sanno di esserlo, in quanto sono quei nodi che hanno l'insieme $|\text{tree-neig}(x)| = 1$, cioè con solo un nodo, mentre i nodi interni sanno il numero di figli, perché mantengono a loro volta la variabile $\text{tree-neig}(x)$, e, per costruzione di tale insieme, il numero di figli di un nodo intero x è pari a $|\text{tree-neig}(x)| - 1$.



Quando la sorgente s riceve il messaggio S da ogni figlio, allora sa che lo spanning tree

è stato costruito e può iniziare una operazione di broadcast per informare tutti i nodi che l'albero ricoprente è stato finalizzato.
Il costo totale per questo processo è quindi pari a

$$2(n - 1)$$

Considerazioni finali

Seguono alcune osservazioni finali sulla costruzione distribuita dello spanning tree appena trattato.

Spanning Tree tramite Broadcast

Si osserva che un qualunque protocollo di broadcast induce in modo implicito un'albero ricoprente T della rete. Tale albero T è ottenuto definendo la relazione padre-figlio seguente

Il padre di un nodo x è il nodo che manda per primo l'informazione ad x .

Questo non è sufficiente per risolvere il problema in quanto i nodi conoscono solo chi è il proprio padre e non chi sono i propri figli: è una conseguenza del fatto che nel broadcast non sono previsti messaggi di ack.

Lower bound per la costruzione dello Spanning Tree

Si osserva che non è possibile definire un protocollo che costruisce uno Spanning Tree utilizzando $o(m)$ messaggi. Infatti per costruire l'albero ricoprente è necessario quantomeno informare tutti i nodi della rete tramite broadcast che il protocollo è iniziato: se così non fosse, alcuni nodi verrebbero esclusi dall'albero, che non risulterebbe ricoprente. Se fosse possibile costruire un'albero ricoprente con meno di m messaggi, allora anche la fase di broadcast richiederebbe meno di m messaggi. Ma questo non è possibile sotto le assunzioni standard.

Che albero costruisce il protocollo?

Si osserva che in generale, fissato un protocollo P che risolve il problema della costruzione distribuita di un'albero ricoprente, lo specifico albero costruito dipende:

- dal protocollo P ;
- dalla particolare esecuzione del protocollo P , ovvero dallo specifico delay dei messaggi in una esecuzione.

Ad esempio il protocollo SHOUT+ è in grado di generare ogni possibile albero ricoprente di G semplicemente cambiando i ritardi dei messaggi. Una conseguenza di questo fatto è che alcune proprietà del grafo G potrebbero essere perse nel particolare spanning tree costruito. Ad esempio si potrebbe avere che, dato T l'albero ricoprente costruito da P :

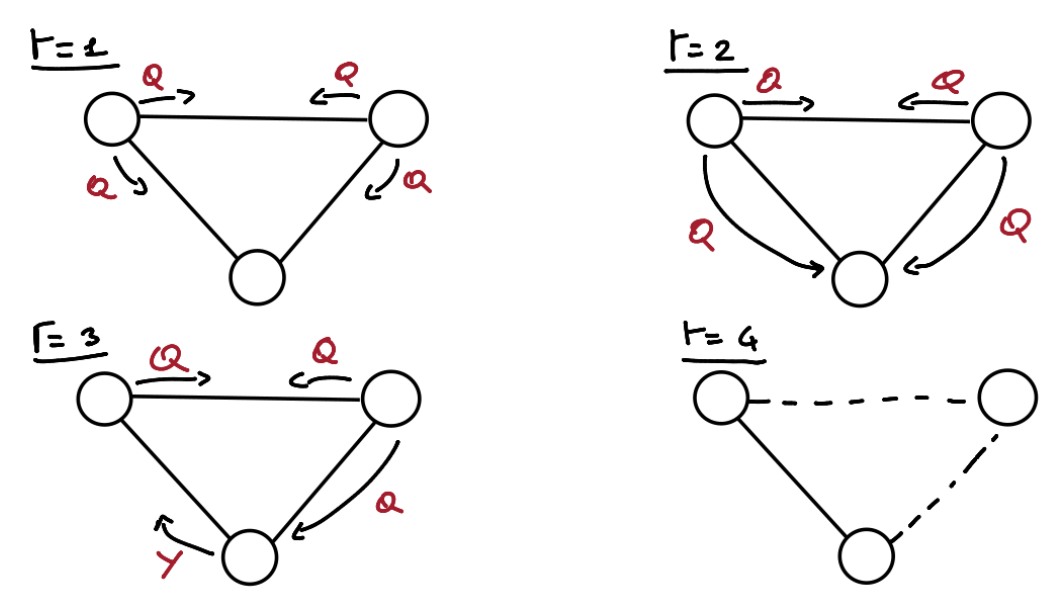
$$\text{diam}(T) \gg \text{diam}(G)$$

Si osserva infine che lavorando in un modello sincrono è possibile simulare una **BFS (Breadth First Search)** per costruire uno spanning tree che rispetta le distanze relative.

Costruzione dello spanning tree con più iniziatori

Ci si chiede se è possibile definire un protocollo che costruisce uno spanning tree anche quando ci sono più initiators.

Nel caso di SHOUT+ non è possibile, come mostra il seguente esempio di esecuzione



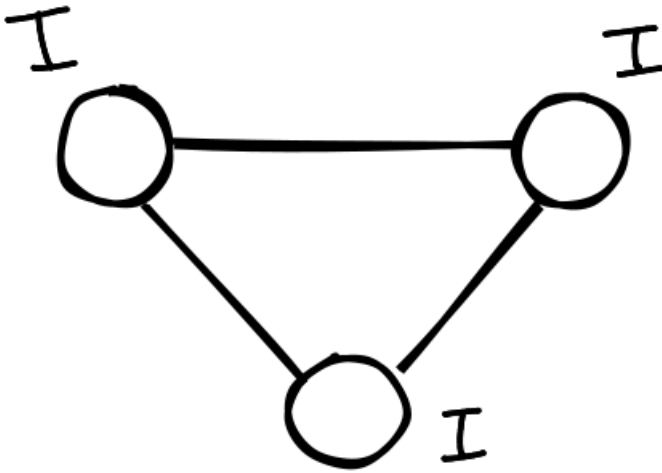
In particolare si dimostra che questo non è un caso particolare per lo SHOUT+, ma è un risultato generale.

Teorema

Il problema della costruzione distribuita di uno spanning tree non è risolubile in modo deterministico sotto le ipotesi generali con più initiators.

Dimostrazione

Si consideri la seguente situazione completamente simmetrica in cui c'è un triangolo con tre iniziatori.



Si ricorda che presi dei nodi aventi lo stesso *stato interno* $\sigma(v, t)$, allora essi da quel momento in poi si comporteranno alla stessa maniera, dato che non sono in grado di distinguere una loro *identità* all'interno della rete.

Allora non si arriverà **mai** in uno stato in cui un nodo è padre e gli altri due figli, in quanto questo implicherebbe che uno dei nodi si è comportato in maniera differente dagli altri due.

Per risolvere questo problema è necessario risolvere un'altro problema: il problema della **leader election**. Eletto un leader tra tutti i nodi initiator, esso potrà eseguire uno dei protocolli di costruzione di spanning tree a singola sorgente già noti.

Si vedrà che il problema della leader election può essere risolto in maniera deterministica solamente se i nodi sono **univocamente identificati** in maniera globale.